

Database Normalization: TournaPro Schema

$$UNF \rightarrow 1NF \rightarrow 2NF \rightarrow 3NF \rightarrow BCNF$$

1. Unnormalized Form (UNF)

The starting point is a single flat table called MatchReport that stores all tournament data in one place. It has three major problems:

- Repeating column groups for players (player1_name, player2_name, ...) — not atomic
- Repeating column groups for goals (goal1_player, goal2_player, ...) — variable-length data in columns
- Standings data duplicated in every match row — no clean primary key

MatchReport (flat god-table)
match_id (PK)
tournament_name, tournament_season, tournament_start, tournament_end
home_team_name, home_team_color, home_team_group
away_team_name, away_team_color, away_team_group
venue_name, venue_city, venue_capacity, venue_surface
home_player1_name, home_player1_jersey, home_player1_pos, home_player2_name... (REPEATING)
goal1_player, goal1_minute, goal2_player, goal2_minute... (REPEATING)
home_goals, away_goals, referee_name
standings_played, standings_won... (duplicated per match)

2. First Normal Form (1NF)

To reach 1NF, we flatten the repeating groups into separate rows, ensure every cell is atomic, and define primary keys. However, partial dependencies remain — some non-key columns depend only on part of a composite key, not the whole key.

What changes from UNF to 1NF

- Repeating player columns are split into a Match_Player junction table with composite PK (match_id, player_id)
- Repeating goal columns are split into a Match_Goal table with composite PK (match_id, goal_id)
- Match information is isolated into Match_Info with composite PK (match_id, tournament_id)

Remaining violations (partial dependencies)

Match_Player

Column	Depends On	Issue
match_id (PK)	(match_id, player_id)	Part of composite key
player_id (PK)	(match_id, player_id)	Part of composite key
player_name	player_id only	<i>partial dependency</i>
jersey_no	player_id only	<i>partial dependency</i>
position	player_id only	<i>partial dependency</i>
team_name	player_id only	<i>partial dependency</i>

Match_Goal

Column	Depends On	Issue
match_id (PK)	(match_id, goal_id)	Part of composite key
goal_id (PK)	(match_id, goal_id)	Part of composite key
minute	(match_id, goal_id)	OK
is_own_goal	(match_id, goal_id)	OK
player_name	player_id only	<i>partial dependency</i>
team_name	player's team only	<i>partial dependency</i>

Match_Info

Column	Depends On	Issue
match_id (PK)	(match_id, tournament_id)	Part of composite key
tournament_id (PK)	(match_id, tournament_id)	Part of composite key

tournament_name	tournament_id only	<i>partial dependency</i>
venue_city	venue_id only	<i>partial dependency</i>
home_team_color	home_team_id only	<i>partial dependency</i>
match_date, status	(match_id, tournament_id)	OK

Rule violated: In 1NF tables with composite keys, every non-key attribute must depend on the WHOLE key, not just part of it. player_name depends only on player_id, not on (match_id, player_id).

3. Second Normal Form (2NF)

To reach 2NF, we eliminate all partial dependencies by extracting entities into their own tables. Each entity gets a single-column surrogate PK. However, transitive dependencies now remain — a non-key column depends on another non-key column rather than directly on the PK.

What changes from 1NF to 2NF

- Player attributes extracted into a standalone Player table (player_id PK)
- Tournament attributes extracted into a Tournament table (tournament_id PK)
- Venue attributes extracted into a Venue table (venue_id PK)
- Team attributes extracted into a Team table (team_id PK)

Remaining violations (transitive dependencies)

Player

Column	Notes
player_id (PK)	Unique identifier
name, jersey_no, position	Directly depend on player_id
team_id (FK)	FK to Team table
team_color (via team_id)	<i>Violation: transitive dependency</i>
team_group (via team_id)	<i>Violation: transitive dependency</i>

T_Match

Column	Notes
match_id (PK)	Unique identifier
tournament_id, home_team_id, away_team_id	Foreign keys
venue_id (FK)	FK to Venue table
venue_city (via venue_id)	<i>Violation: transitive dependency</i>
venue_capacity (via venue_id)	<i>Violation: transitive dependency</i>
match_date, match_time, status	Directly depend on match_id

Result

Column	Notes
result_id (PK)	Unique identifier
match_id (FK, UNIQUE)	One result per match
home_goals, away_goals, recorded_at	Directly depend on result_id
recorded_by (FK)	FK to User table
referee_name (via recorded_by)	<i>Violation: transitive dependency</i>

Standings

Column	Notes
(tournament_id, team_id) PK	Composite primary key
played, won, drawn, lost	Directly depend on PK
goals_for, goals_against	Directly depend on PK
goal_diff (computed)	Violation: derived from other columns
points (computed)	Violation: derived from other columns

Rule violated: In 2NF tables, no non-key attribute may depend on another non-key attribute. player_id -> team_id -> team_color is a transitive chain — team_color should live only in Team.

4. Third Normal Form (3NF) — Final Schema

To reach 3NF, we eliminate all transitive dependencies. Every non-key attribute now depends directly and only on the primary key of its table. Computed columns (goal_diff, points) are made GENERATED ALWAYS AS so they are not independently stored.

What changes from 2NF to 3NF

- Team attributes (color, group_name) live only in Team — no more player -> team_id -> color chain
- Venue attributes (city, capacity) live only in Venue — T_Match only stores the FK
- Referee info lives in User — Result only stores the FK recorded_by
- goal_diff and points are GENERATED ALWAYS AS expressions — not redundantly stored values

Clean 3NF Tables

Tournament

Column	Type	Key	Notes
tournament_id	INT	PK	Auto increment
name	VARCHAR(100)		
season	VARCHAR(20)		
start_date, end_date	DATE		
status	ENUM		upcoming/active/completed

Venue

Column	Type	Key	Notes
venue_id	INT	PK	Auto increment
name	VARCHAR(100)		
city	VARCHAR(50)		
capacity	INT		
surface	ENUM		Natural/Hybrid/Artificial

Team

Column	Type	Key	Notes
team_id	INT	PK	Auto increment
name	VARCHAR(100)	UNIQUE	
color	VARCHAR(10)		
group_name	CHAR(1)		A or B

Player

Column	Type	Key	Notes
player_id	INT	PK	Auto increment
name	VARCHAR(100)		
jersey_no	INT		
position	ENUM		FW/MF/DF/GK
team_id	INT	FK -> Team	

User

Column	Type	Key	Notes
user_id	INT	PK	Auto increment
name	VARCHAR(100)		
password_hash	VARCHAR(255)		
role	ENUM		admin/referee/captain
team_id	INT	FK -> Team	Nullable, captains only

T_Match

Column	Type	Key	Notes
match_id	INT	PK	Auto increment
tournament_id	INT	FK -> Tournament	

home_team_id	INT	FK -> Team	
away_team_id	INT	FK -> Team	
venue_id	INT	FK -> Venue	
match_date	DATE		
match_time	TIME		Nullable
status	ENUM		upcoming/live/playe d

Result

Column	Type	Key	Notes
result_id	INT	PK	Auto increment
match_id	INT	FK -> T_Match, UNIQUE	One result per match
home_goals	INT		Default 0
away_goals	INT		Default 0
recorded_by	INT	FK -> User	referee/admin
recorded_at	TIMESTAMP		Auto set

Standings

Column	Type	Key	Notes
tournament_id	INT	PK (composite)	FK -> Tournament
team_id	INT	PK (composite)	FK -> Team
played, won, drawn, lost	INT		Default 0
goals_for, goals_against	INT		Default 0
goal_diff	INT	GENERATED	goals_for - goals_against
points	INT	GENERATED	won*3 + drawn

Goal

Column	Type	Key	Notes
goal_id	INT	PK	Auto increment
match_id	INT	FK -> T_Match	
player_id	INT	FK -> Player	
minute	INT		Nullable
is_own_goal	BOOLEAN		Default FALSE
is_assist	BOOLEAN		Default FALSE

Tournament_Team

Column	Type	Key	Notes
tournament_id	INT	PK (composite)	FK -> Tournament
team_id	INT	PK (composite)	FK -> Team

5. Boyce-Codd Normal Form (BCNF)

BCNF is a stricter version of 3NF. The rule is: for every functional dependency $X \rightarrow Y$ in a table, X must be a superkey (i.e., X alone must uniquely identify every row).

Difference between 3NF and BCNF

3NF allows one exception: a non-key attribute can determine another attribute if it is part of a candidate key. BCNF closes this loophole entirely — no exceptions. This gap only matters when a table has multiple overlapping candidate keys.

Why a BCNF violation would occur (example)

Imagine adding `coach_name` to the `Player` table. Since `coach_name` depends on `team_id` (not on `player_id`), and `team_id` is not a superkey of `Player`, this would be a BCNF violation even if it were technically 3NF-safe in some edge cases. The fix is what was already done: keep coach-level data in `Team`.

BCNF Analysis of TournaPro Schema

Table	Candidate Keys	Any Non-Superkey Determinant?	BCNF?
Tournament	tournament_id	No	Yes
Venue	venue_id	No	Yes
Team	team_id, name (UNIQUE)	No — name determines same attrrs as team_id	Yes
Player	player_id, (team_id, jersey_no)	No — both are superkeys	Yes
User	user_id	No	Yes
T_Match	match_id	No	Yes
Result	result_id, match_id (UNIQUE)	No — both are superkeys	Yes
Standings	(tournament_id, team_id)	No	Yes
Goal	goal_id	No	Yes
Tournament_Team	(tournament_id, team_id)	No	Yes

In short: the normalization journey UNF -> 1NF -> 2NF -> 3NF was the real work. BCNF came for free because of good design choices — single-purpose tables, surrogate PKs, and no overlapping candidate key conflicts.